CENTRO UNIVERSITÁRIO IESB
Centro Universitário Instituto de Educação Superior de Brasília
Coordenação de Engenharia / Ciência da Computação

ANÁLISE COMPARATIVA DO DESEMPENHO DE COMPILADORES EM ALGORITMOS DE REGRESSÃO LINEAR

Dannyel Fontenele Ribeiro[1] e Roger Rocha Ferreira[2]

RESUMO: Este trabalho compara, em um processador AMD Zen 5, a resolução de mínimos quadrados ordinários por equações normais (NE), implementadas de forma ingênua e compiladas com GCC e Clang em O0, O1 e O3 com -march=native, e pela decomposição QR de Householder da biblioteca OpenBLAS, usada como referência, avaliadas em conjuntos sintéticos de 10^3 a 10^6 amostras e de 11 a 101 variáveis, usando a mediana de 20 execuções. A estabilidade foi medida com uma solução de referência com precisão estendida, com números de condição entre 10 e 10^8. O QR foi mais rápido que todas as configurações de NE, de 4,5x-12,2x em O0 a 2,1-3,8x em O1 e O3, com mínimo de 1,1x em p = 11. O ganho concentrou-se na transição de O0 para O1, O3 -march=native não alterou o desempenho no GCC e foi irregular no Clang, e nenhum dos dois vetorizou a formação de X^TX ou X^Ty. O erro de NE cresceu com o quadrado do número de condição, e sua fatoração de Cholesky falhou acima de cerca de $5 \cdot 10^7$, limitando-se a 3% de erro antes da falha, enquanto QR cresceu linearmente com o número, sem superar $3 \cdot 10^{-9}$. Os resíduos foram praticamente os mesmos em todas as resoluções.

Palavras-chave: Regressão linear, otimização de compiladores, OpenBLAS, equações normais, decomposição QR.

1 INTRODUÇÃO

O desempenho computacional constitui um aspecto relevante na execução de algoritmos numéricos e científicos, especialmente em aplicações de aprendizado de máquina, cuja evolução tem sido acompanhada por uma crescente demanda por recursos computacionais (Thompson et al., 2020). Nesse contexto, a regressão linear pelo método dos mínimos quadrados ordinários representa um problema adequado para investigar a relação entre a implementação de algoritmos, a eficiência de sua execução, e a precisão dos resultados, permitindo contrastar diferentes métodos numéricos e suas características. Além da escolha do método numérico, o desempenho de um programa compilado pode ser influenciado pelas otimizações realizadas pelo compilador. Os compiladores otimizadores, como o GCC e o Clang, conseguem analisar o código-fonte e aplicar transformações como *inlining* de funções e vetorização ou desenrolamento de laços para melhorar o tempo de execução do programa. Além disso, determinadas opções de compilação permitem sacrificar a conformidade estrita com o padrão IEEE 754 para aritmética de ponto flutuante (IEEE, 2019) em troca de ganhos de velocidade, o que pode causar perda de precisão dos resultados dos cálculos (GNU PROJECT, 2026).

Logo, há a necessidade de avaliar o quanto, para uma dada arquitetura, compiladores modernos conseguem reduzir a diferença de desempenho entre uma implementação própria, cujas otimizações dependem do compilador, e uma implementação de biblioteca especializada, que pode explorar de maneira mais eficiente os recursos do *hardware*

[1] Graduando em Ciência da Computação. E-mail: dannyelfr@yahoo.com.br
[2] Esp. E-mail: uni.pro.frogger@gmail.com

disponível. Para esse estudo de caso, será avaliado o desempenho dos compiladores GCC e Clang, para linguagem C, na otimização da regressão linear pelo método das equações normais a diferentes níveis de otimização, tomando-se como base de comparação operações LAPACK da biblioteca OpenBLAS, como a decomposição QR de Householder, na plataforma AMD Zen 5, em ambiente Linux, conforme variam os tamanhos de entrada. Como se trata de uma avaliação de métodos numéricos, além do tempo de execução, para cada configuração de teste, tanto a precisão das previsões e dos coeficientes quanto a estabilidade numérica dos métodos empregados serão escrutinadas, para determinar uma possível associação entre a velocidade da implementação e a qualidade de seus resultados. A relevância deste estudo está na investigação dos limites das otimizações automáticas de compiladores diante de implementações numéricas especializadas.

O objetivo deste trabalho é avaliar o desempenho de uma implementação de regressão linear por equações normais compilada com GCC e Clang em diferentes níveis de otimização, utilizando a decomposição QR de Householder da OpenBLAS como referência. Para isso, serão comparados os tempos de execução e a qualidade numérica das soluções para diferentes tamanhos de entrada, considerando também as otimizações aplicadas pelos compiladores e os recursos da arquitetura AMD Zen 5. O restante deste trabalho está organizado da seguinte forma: a Seção 2 apresenta o referencial teórico e estudos correlatos; a Seção 3 descreve a metodologia experimental; a Seção 4 apresenta e discute os resultados; e a Seção 5 reúne as conclusões do estudo.

2 REFERENCIAL TEÓRICO

2.1 Regressão Linear e seus Métodos

Os dois métodos a serem comparados têm como objetivo estimar uma série de coeficientes β tal que, multiplicados para uma série de valores em X, aproxime o valor respectivo em y, tal que se minimize a soma dos quadrados da diferença entre o valor real e o valor previsto (Chein, 2019). Para tanto, existem algumas maneiras de determinar os coeficientes, e o método mais ensinado, e um dos dois a ser comparado, é por equações normais, na qual determina-se o vetor de coeficientes pela resolução da equação $X^T X\beta = X^T y$. Porém, a formação de $X^T X$ eleva o número de condicionamento ao quadrado, isto é, a sua sensibilidade a erros (Higham, 2020a), o que torna o método de equações normais menos numericamente estável, o que faz com que erros pequenos possam aumentar significativamente e distorcer o resultado dos coeficientes (Higham, 2020b).

Por isso, o método de decomposição QR por transformações de Householder torna-se mais atraente, pois não há formação explícita de $X^T X$, e a matriz X é decomposta em uma matriz ortonormal Q e uma triangular superior R, tal que, ao se desenvolver as substituições, o sistema linear a ser resolvido torna-se $R\beta = Q^T y$. Essa decomposição é usada na implementação de regressores lineares, por exemplo, na linguagem R, com a função lm (R Core Team, s.d.), ou em LAPACK com a família XGELS de funções, como DGELS (LAPACK, s.d.). Apesar disso, esse método pode ser mais lento do que por equações normais por exigir mais operações aritméticas em certas ocasiões (Golub; Van Loan, 2013).

2.2 Compiladores, Níveis de Otimização, e Bibliotecas Especializadas

O GCC e o Clang estão entre os compiladores de linguagem C mais populares (Jetbrains, 2025), possuindo opções, ou *flags*, de otimização tanto para velocidade quanto para tamanho do executável. Algumas delas são agrupadas sob as opções -O, que variam de -O0, que é a padrão e desativa as otimizações, até -

O3, que habilita otimizações mais agressivas para tentar aumentar a velocidade do programa (a custo de velocidade de compilação), como a permuta de laços (*loop interchange*) ou a vetorização dos mesmos (GNU Project, [s. d.]; LLVM Project, [s. d.]). Os recursos de arquitetura específicos da CPU alvo, como extensões AVX-512, podem ser usados, em conjunto com a opção -O3, por meio da opção -march=native, para permitir mais algumas otimizações, em troca de portabilidade.

Apesar disso, muito ainda depende da implementação algorítmica e, para esse fim, bibliotecas como a OpenBLAS, que implementa rotinas das especificações BLAS e LAPACK, fornecem funções altamente otimizadas para a realização de operações envolvendo álgebra linear. Diferentemente de uma implementação própria, essas bibliotecas podem explorar otimizações específicas para determinadas arquiteturas de processadores, incluindo o uso de instruções vetoriais e técnicas de paralelização, além de estratégias que reduzem o acesso à memória e melhoram o aproveitamento da hierarquia de *cache*. Dessa forma, podem alcançar níveis de desempenho difíceis de reproduzir apenas com as otimizações automáticas de um compilador. A OpenBLAS oferece rotinas LAPACK otimizadas para a resolução de sistemas de mínimos quadrados pela decomposição QR e solucionadores, usando a interface C LAPACKE (OPENBLAS, [s.d.]).

2.3 Arquitetura AMD Zen 5

Para a realização deste trabalho, é importante utilizar uma arquitetura moderna, com recursos relevantes para a execução de aplicações numéricas. Nesse contexto, a microarquitetura AMD Zen 5, lançada em 2024 para processadores de computadores pessoais e servidores, atende a esses requisitos. Entre suas características, destacam-se a capacidade de prever até dois desvios tomados por ciclo de *clock*, seis unidades lógicas aritméticas para operações inteiras e, por núcleo, 80 KB de *cache* L1, 48 KB dos quais para dados, e 1MB de *cache* L2, com *cache* L3 global mínimo de 32 MB. Além disso, a arquitetura possui suporte a AVX-512 com caminhos de dados de 512 bits para ponto flutuante, permitindo a execução de operações vetoriais com maior largura de dados (AMD, 2024). Essa última característica é de bastante interesse para os experimentos de otimização, visto que ao menos parte dos laços poderão ser vetorizados, possivelmente com instruções dessa extensão. Além disso, a capacidade das memórias *cache* da arquitetura permite manter uma parcela significativa dos conjuntos de dados, especialmente os menores, próximos às unidades de processamento, reduzindo a necessidade de acessos à memória principal. Esse aspecto pode favorecer o desempenho das operações matriciais.

2.4 Trabalhos Correlatos

Larabel (2026) comparou o desempenho dos compiladores GCC 16 e LLVM/Clang 22 utilizando um processador AMD Ryzen Threadripper 9980X, baseado na arquitetura Zen 5, com as opções de otimização -O3 -march=native. Os testes abrangeram diferentes aplicações, como processamento de imagens, codificação de vídeo, rastreamento de raios e processamento digital de sinais. Os resultados mostraram diferenças de desempenho entre os compiladores, sem que um deles apresentasse vantagem em todas as aplicações avaliadas. Na biblioteca Liquid-DSP, por exemplo, o Clang obteve resultados superiores em diversas configurações. Essa variação mostra que a escolha do compilador pode influenciar o desempenho de acordo com as características da aplicação, algo possivelmente relevante para os experimentos com algoritmos numéricos deste trabalho.

Relacionado ao desempenho de compiladores, Feng, He e Tao (2021) investigaram, com o benchmark TSVC (Test Suite for Vectorizing Compilers), os limites dos

compiladores ICC, GCC e Clang quanto à capacidade de vetorizar e a melhoria de desempenho após a vetorização. Descobriram que, para essa suíte, o GCC (versões 5.5.0 a 9.2.0) conseguiu vetorizar 43,7% dos testes, enquanto o Clang (versões 5.0.1 a 9.0.0) conseguiu 32,4%, e que a capacidade de vetorização depende, dentre outros fatores, da existência de desvios condicionais e de dependências entre arrays e entre iterações, e, por isso, operações de redução ou programas sem dependências impeditivas são mais propensos a serem vetorizados, e algumas transformações como a permuta de laços pode ajudar facilitar essa operação. Portanto, características dos laços e análise de dependência influenciam na capacidade de vetorização dos compiladores.

Chang, Kang e Titley-Peloquin (2020) reafirmam Baboulin et al (2009) ao afirmarem que as equações normais podem exigir até aproximadamente metade das operações de ponto flutuante comparado à decomposição QR quando o número de observações é muito maior que o número de variáveis. Em contrapartida, elas possuem maior sensibilidade a erros pois dependem do quadrado do nível de condicionamento, enquanto o método por fatoração QR possui dependência linear. Logo, há necessidade de ponderar custo computacional e precisão da solução ao escolher a implementação da regressão linear.

2.5 Síntese do Referencial

Os estudos indicam que o desempenho da regressão linear depende do método numérico empregado e das possíveis otimizações pelo compilador, que pode ou não aproveitar recursos do hardware. As equações normais são menos custosas computacionalmente, porém são menos estáveis, enquanto a decomposição QR possui características opostas. A escolha de compilador, entre GCC e Clang, pode influenciar na performance de uma mesma implementação devido à capacidade de otimizá-la, por reordenação e vetorização dos laços ou uma melhor análise de dependência, por exemplo. Como a OpenBLAS, uma biblioteca madura, oferece a decomposição QR, que pode ser usada como implementação de referência, e que recursos da arquitetura AMD Zen 5 permitem possibilidades adicionais de otimização pelos compiladores e que uma parcela considerável dos dados estejam em *cache*.

3 METODOLOGIA

3.1 Ambiente Experimental

Os experimentos foram realizados em um computador com um AMD Ryzen 9 9950X3D, da arquitetura Zen 5, no qual foi feito um *undervolt* global de 20 mV, e 128 GB de RAM DDR5, com clock de 6400 MT/s e latência CL 42. Quanto ao ambiente de execução, usou-se o Ubuntu 26.04.1 LTS, com kernel 6.18.33.2-microsoft-standard-WSL2, no WSL 2 (Subsistema Linux para Windows), e plano de energia "Performance" nas configurações do Windows 11 25H2. Para criação dos experimentos, usou-se o Clang 21 1:21.1.8-6ubuntu1, o GCC 15 15.2.0-16ubuntu1, GNU Make 4.4.1-3, liblapacke 3.12.1-7ubuntu1, e libopenblas-dev 0.3.32+ds-5.

Para o experimento principal, usou-se double como tipo de dado para os valores das matrizes, dispostas em ordem de linha, com ponteiro restrict, pois pode ajudar o compilador a realizar algumas otimizações como reordenar operações ou permitir vetorização, enquanto o experimento de estabilidade usa long double para a solução de referência com QR de Householder, separada do OpenBLAS, para o cálculo mais preciso de erros. Para as opções de compilação, habilitaram-se relatórios de vetorização e substituição do nome da função de regressão por equações normais para cada configuração, para que todas as versões fossem acessíveis ao mesmo tempo.

Os dados foram gerados de forma sintética e pseudoaleatória, de modo a modelar um problema de regressão linear múltipla $y = X\beta + \varepsilon$. Utilizou-se o gerador xoshiro256** (Blackman; Vigna, 2021) com semente fixa (42), inicializado por splitmix64 (Steele Jr.; Lea; Flood, 2014), o que torna os conjuntos reprodutíveis. Para cada número de variáveis independentes $p \in \{10,20,\ldots,100\}$, acrescido do intercepto (coluna de uns), foi gerada uma matriz de projeto com 10^6 amostras e um vetor de coeficientes β com entradas uniformes em $[0,1)$; a resposta foi calculada como $y = X\beta + \varepsilon$, com ε uniforme em $[-10^{-3}, 10^{-3})$. Dois tipos de conjunto foram produzidos: bem condicionado, com colunas independentes e uniformes em $[0,1)$ (número de condição $\kappa_2(X)$ entre 12 e 95), e mal condicionado, em que cada coluna de índice par é a coluna anterior acrescida de ruído uniforme em $[-10^{-3}, 10^{-3})$, resultando em colunas quase colineares ($\kappa_2(X)$ entre aproximadamente $4{,}8 \cdot 10^3$ e $1{,}6 \cdot 10^4$). Os subconjuntos de 10^3, 10^4, 10^5 e 10^6 amostras correspondem às primeiras linhas de cada arquivo.

Para o experimento de estabilidade numérica, foram gerados conjuntos adicionais com 10^4 amostras e $p \in \{11, 51, 101\}$, variando a amplitude do ruído de colinearidade em 10^{-3}, 10^{-5}, 10^{-6} e 10^{-7}, além do caso bem condicionado, o que cobre $\kappa_2(X)$ de cerca de 10 a 10^8. Os dados são gravados em arquivos binários (número de amostras, número de variáveis, β, y e X em ordem de linhas) antes das medições e carregados em memória antes de cada bloco de testes.

3.2 Implementações

As implementações usadas no experimento principal foram o método de equações normais, doravante denominadas NE para *Normal Equations*, com formação explícita de X^TX e X^Ty, e uma que utiliza funções LAPACK da biblioteca OpenBLAS, que serviu como base para comparações. O primeiro método usou decomposição de Cholesky para se criar uma matriz triangular inferior, e substituição progressiva e regressiva para determinar o vetor de coeficientes. Já pelo segundo método, fez-se a decomposição QR de Householder da matriz X, seguida de uma sobrescrita pela multiplicação da transposta da matriz ortogonal Q ($y \leftarrow Q^Ty$) e, por fim, a resolução do sistema triangular resultante ($R\beta = Q^Ty$), que retorna o vetor de coeficientes, e será referido monominamente como QR. Em ambos os casos, faz-se a medição de tempo de cada etapa e determina-se os resíduos $y_{true} - y_{pred}$. Como LAPACKE atua em ordem de coluna e o *dataset* está em ordem de linha, o tempo de QR inclui a transposição interna entre as ordens.

A implementação por equações normais foi compilada seis vezes, com GCC e Clang e três níveis de otimização conforme a tabela a seguir, e QR obtido da biblioteca OpenBLAS, com código pré-compilado:

Tabela 1 – Configurações

Método	Compilador	Flags
NE	GCC	-O0
NE	GCC	-O1
NE	GCC	-O3 -march=native
NE	Clang	-O0
NE	Clang	-O1
NE	Clang	-O3 -march=native
QR	GCC	-O0

Fonte: Elaborado pelo autor

Para realizar comparações de forma justa, todas as configurações usam somente uma *thread*, e, para evitar distorções de ponto flutuante, a *flag* -ffast-math foi omitida. A implementação em OpenBLAS foi compilada somente com GCC em nível O0, e para evitar muitas faltas de página durante as transposições do LAPACKE, desativou-se o mmap pelo alocador do glibc para alocações grandes com MALLOC_MMAP_MAX_=0 e impediu-se que ele devolvesse memória ao SO após free

com MALLOC_TRIM_THRESHOLD_ com o valor de 4GiB (Kerrisk, 2026).

3.3 Métricas e Procedimentos de Análise

Para cada configuração, que foram compiladas com emissão de relatórios de vetorização, fez-se 3 rodadas de aquecimento e 20 rodadas de medição, na qual foram medidos o número de condicionamento, tempo de formação das matrizes X^TX e X^Ty (no caso de NE), tempo de decomposição (por método de Cholesky ou por QR), tempo de solução após a decomposição, as normas dos erros dos coeficientes e dos resíduos, tanto absolutos quanto relativos aos valores verdadeiros, o erro absoluto médio e a raíz do erro quadrático médio deles, e o mínimo e máximos absolutos.

Com essas métricas, o *gap* das 6 configurações relativo à implementação com OpenBLAS pôde ser determinado, assim como as outras métricas puderam ser analisadas em Python com as bibliotecas Matplotlib, Seaborn e pandas. Como as compilações do método de equações normais possuíam relatórios de vetorização, eles também foram analisados.

4 RESULTADOS E DISCUSSÃO

Os resultados a seguir referem-se à mediana de 20 execuções por combinação de método, tamanho amostral (n = 10^3 a 10^6) e número de variáveis (p = 11 a 101)[1]. Os tempos medidos nos conjuntos bem condicionados ($\kappa_2 \leq 95$) e mal condicionados ($\kappa_2 \leq 1{,}6 \cdot 10^4$) diferiram entre 0,972 e 1,051 vezes para o QR e entre 0,81 e 1,11 vezes para o NE, sendo os desvios maiores restritos a n = 10^3, onde o ruído é mais alto. Por isso, as figuras mostram apenas o conjunto bem condicionado. Nesses conjuntos, NE e QR produzem soluções equivalentes (coeficientes de variância com diferença relativa de até 0,09%), de modo que a comparação de tempo se dá entre métodos que entregam o mesmo resultado.

4.1 Desempenho

A Figura 1 mostra a razão de tempo entre NE e QR para cada variante, n e p.

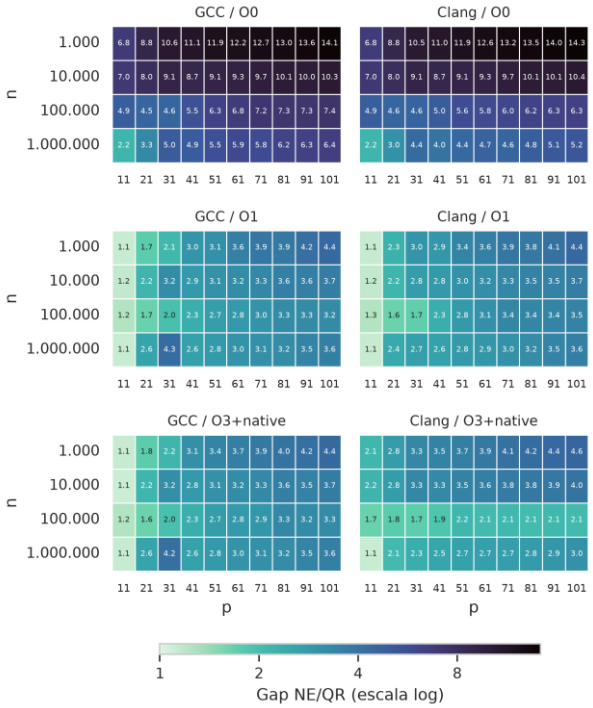


Figura 1 – *Gaps* de Desempenho (cor em escala $\log_{10}$)

Tomando-se a mediana sobre p, NE não otimizado foi 4,5 a 12,2 vezes mais lento que QR, em -O1 foi 2,8 a 3,5 vezes, e, em O3 com -march=native, entre 2,1 e 3,8. Entre os compiladores, houve alguns casos de diferença, sendo o mais extremo no *native* em n = 10^6 e p = 31, com uma diferença de 4,2 no GCC contra 2,3 no Clang, cuja causa do GCC não foi identificada. Percebe-se que a lacuna depende mais de p do que de n, visto que em *native*, com n = 10^6, a razão com p = 11 é 1,1 em ambos os compiladores, porém 3 e 3,6 em p = 101 no Clang e GCC, respectivamente. Embora NE realize em torno da metade das operações de QR, conforme Baboulin et al. (2009), ele não foi mais rápido, sugerindo que a diferença decorre da taxa de execução das operações, e não da contagem delas.

1 Repositório disponível em https://github.com/DanFonR/tcc-cs-iesb

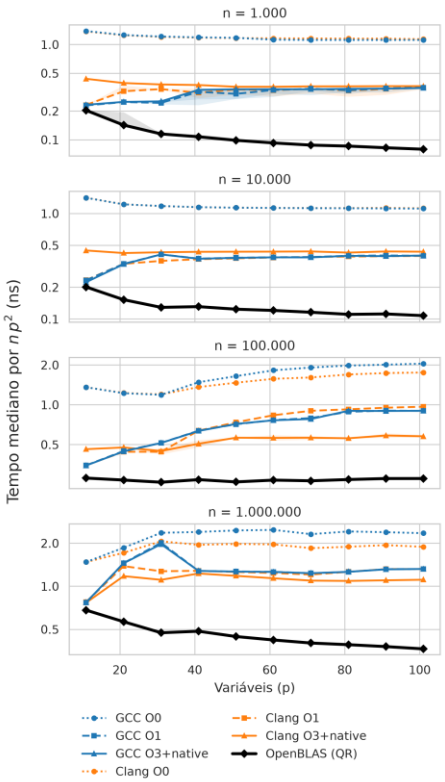


Figura 2 – Tempos medianos normalizados x p

Vê-se pela Figura 2 que entre n = 10^5 e n = 10^6 que o tempo cresceu de 11,6x a 19,9x para um aumento de 10x em n, provavelmente devido ao tamanho do *dataset*, que pode não ter cabido inteiramente em *cache*. Como LAPACKE trabalha naturalmente em ordem de coluna e o *dataset* estava em ordem de linha, os tempos medidos em QR incluem as transposições internas, o que torna a lacuna real entre ele e NE possivelmente ainda maior, o que contraria a expectativa de Golub e Van Loan (2013) que menos operações signifiquem menos tempo.

4.2 Limites dos Compiladores

A Figura 3 mostra que o *speedup* de O0 para O1 foi de aproximadamente 3,5x em n = 10^3 para ambos os compiladores, e 1,88x no GCC e 1,52x no Clang em n = 10^6. Ou seja, o *speedup* diminui conforme n aumenta, provavelmente pois os laços passam a depender de memória.

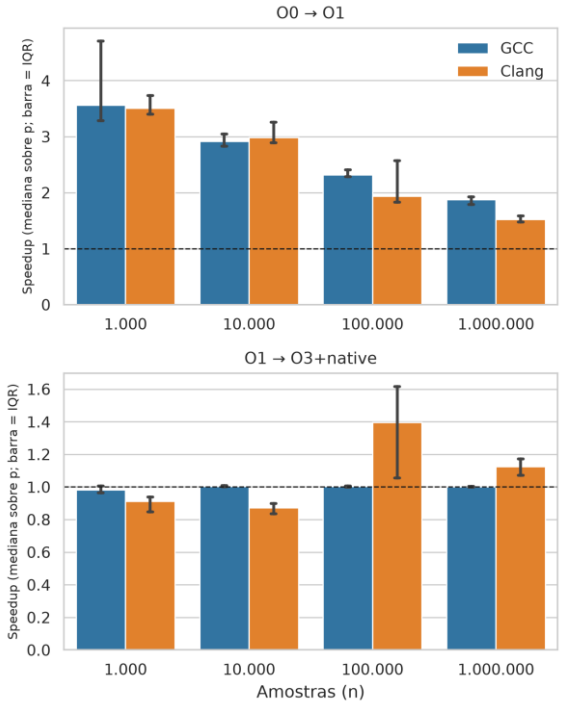


Figura 3 – *Speedup* das transições de otimização

Na transição entre O1 e *native*, o GCC manteve-se aproximadamente em 1x, enquanto Clang variou mais. Essa segunda transição é explorada na Figura 4.

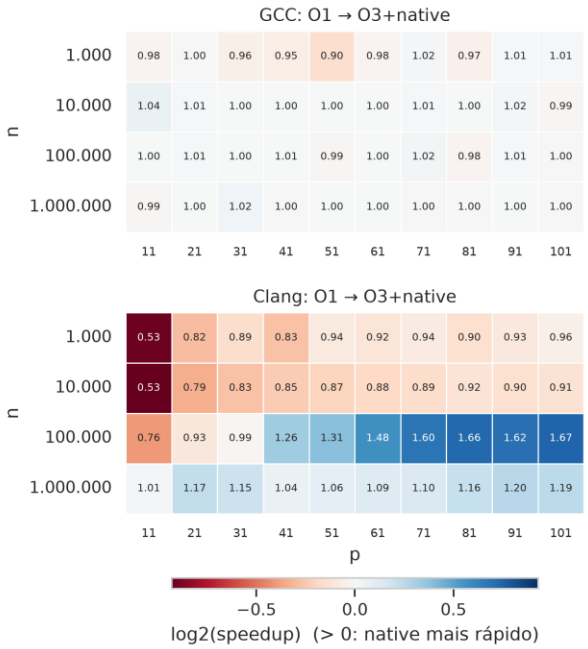


Figura 4 – Mapas de *speedup* entre O1 e *native* (cor em escala $\log_2$)

Conforme ela mostra, os *speedups* com GCC ficaram entre 0,9x e 1,04x, com desvios maiores em n = 10^3, enquanto os do Clang estão dispostos em regiões de maior lentidão, quando n ≤ 10^4, com pior *speedup* de 0,53x em p = 11, de ganho modesto, quando n = 10^6, onde varia entre 1,01x e 1,2x, e de maior rapidez, com n = 10^5 e p ≥ 41, de 1,26x a 1,67x. No GCC, o assembly dos laços de formação de X^TX e X^Ty com *native* tem a mesma estrutura do O1, com multiplicação e soma escalares separadas, sem FMA e sem desenrolamento, o que é compatível com o *speedup* de 1,00x. No Clang, o *native* desenrola o laço de X^TX 8 vezes e usa FMA em uma única cadeia de dependência, diferença que é compatível com o comportamento irregular observado. Não foi realizado estudo de ablação para determinar se os casos pequenos foram prejudicados pela latência da cadeia de FMA, nem se o ganho em n = 10^5 decorre do desenrolamento.

Portanto, pela análise dos gráficos, o ganho vem quase todo da transição entre O0 e O1. O *native*, conforme os relatórios de vetorização de ambos os compiladores, não vetorizou os laços de custo $O(n \cdot p^2)$, somente os laços de decomposição Cholesky ($O(p^3)$) e substituição direta ($O(p^2)$) no GCC. Já no Clang, os ganhos são irregulares, e não houve vetorização.

4.3 Estabilidade numérica

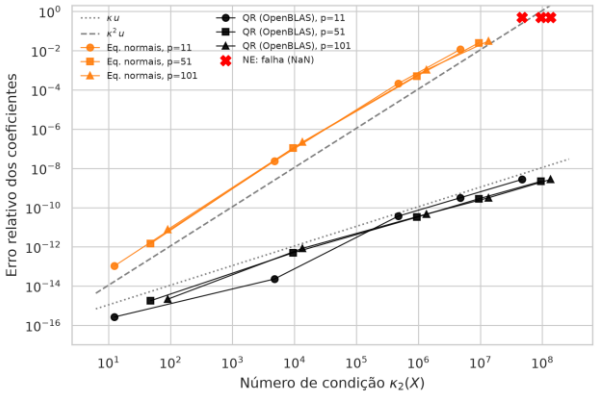


Figura 5 – Relação erro x condição

Conforme a Figura 5, NE acompanha $\kappa^2 u$ (u = 2^{-53}, unidade de arredondamento do ponto flutuante) para todos os valores de p, ficando 1,5 a 11 vezes acima da linha de referência, porém apresenta falha quando κ é muito alto devido a erros de arredondamento de ponto flutuante conforme IEEE 754, o que resulta em X^TX deixar de ser definida positiva, levando a um pivô não positivo na função que implementa Cholesky, que propaga NaN para NE, já que não possui guarda para tal, o que pode ser explorado em trabalhos futuros. Já QR acompanha κu, com a razão erro/κu entre 0,04 e 0,7, tal que o erro fosse menor em p baixo quando κ também é baixo. Como a implementação ingênua de NE resultou em tempos maiores e menor estabilidade, os ganhos teóricos de Chang et. al (2020) não se concretizaram. Ou seja, a vantagem teórica em operações depende da qualidade da implementação.

Porém, conforme mostra a Tabela 2, mesmo com número de condição alto, NE possuiu erro

Tabela 2 – Erros relativos dos coeficientes contra a referência

$\kappa_2(X)$	NE	QR
≤ $9{,}9 \cdot 10^2$	$1{,}1 \cdot 10^{-13}$ a $7{,}6 \cdot 10^{-12}$	$2{,}7 \cdot 10^{-16}$ a $2{,}2 \cdot 10^{-15}$
10^3 a $9{,}9 \cdot 10^4$	$2{,}3 \cdot 10^{-8}$ a $2{,}2 \cdot 10^{-7}$	$2{,}3 \cdot 10^{-14}$ a $8{,}4 \cdot 10^{-13}$
10^5 a $2{,}9 \cdot 10^6$	$2{,}1 \cdot 10^{-4}$ a $1{,}1 \cdot 10^{-3}$	$3{,}3 \cdot 10^{-11}$ a $4{,}7 \cdot 10^{-11}$
$3 \cdot 10^6$ a $2{,}9 \cdot 10^7$	$1{,}2 \cdot 10^{-2}$ a $3{,}1 \cdot 10^{-2}$	$2{,}8 \cdot 10^{-10}$ a $3{,}2 \cdot 10^{-10}$
≥ $3 \cdot 10^7$	Falha (NaN)	$2{,}2 \cdot 10^{-9}$ a $2{,}8 \cdot 10^{-9}$

Fonte: Elaborado pelo autor

máximo de pouco mais de 3%, apesar de apresentar piores resultados que QR, que possuiu erro máximo de $2{,}8 \cdot 10^{-9}$ com κ maior. Além disso, o resíduo relativo entre NE e QR foi menor ou igual a $2{,}3 \cdot 10^{-6}$. As 6 variantes de NE diferiram pouco nos testes de estabilidade.

Essa análise teve limitações, pois o

experimento de estabilidade foi feito com n = 10^4 para dados sintéticos usando um algoritmo ingênuo de equações normais, o que limita a capacidade de vetorização pelos compiladores, e foi realizado em uma única máquina sob WSL2.

5 CONSIDERAÇÕES FINAIS

Os resultados da comparação em um processador AMD Zen 5 das equações normais, implementadas ingenuamente e compiladas em dois compiladores e três níveis de otimização, com a decomposição QR de Householder da biblioteca OpenBLAS mostram que a otimização de compiladores reduz o *gap* entre as implementações, mas não o fecha devido à falta de vetorização dos laços de formação de X^TX e X^Ty nas equações normais, com diferença mínima entre as duas implementações de 1,1x em p = 11, e de cerca de 3x em p = 101.

Quanto ao *speedup* entre compilações, os ganhos se concentram na transição de O0 para O1, enquanto O1 para O3+native não alterou nada no GCC e foi inconsistente no Clang, com zonas de melhora e uma de piora.

A estabilidade das equações normais acompanhou κ^2, com o erro ficando de 1,5 a 11 vezes acima de $\kappa^2 u$, porém a fatoração de Cholesky falhou para κ muito alto. Já o erro de QR manteve-se abaixo de κu, e não superou $3 \cdot 10^{-9}$. Os resíduos das 7 configurações foram praticamente iguais, mesmo com erro de 3% nos coeficientes das equações normais.

Os resultados se limitam a uma máquina com WSL2 e dados sintéticos, com estabilidade avaliada somente em n = 10^4. Trabalhos futuros podem reordenar os laços das equações normais para viabilizar a vetorização, avaliar os impactos da *flag* -ffast-math, isolar o efeito do FMA no Clang, e incluir uma verificação de pivô na decomposição de Cholesky.

ABSTRACT: *This work compares the solution of least ordinary squares through normal equations (NE), with a naïve implementation compiled with GCC and Clang on O0, O1 and O3 with -march=native, with Householder QR decomposition, implemented with OpenBLAS and used as reference, on a synthetic dataset of* 10^3 *to* 10^6 *samples and 11 to 101 variables on an AMD Zen 5 processor, using the median of 20 runs. Stability was measured against a reference solution with extended precision with condition numbers from 10 to* 10^8*. QR was faster than all NE configurations, from 4.5x-12.2x in O0 to 2.1-3.8x in O1 and O3, with minimum 1.1x on p = 11. The gains were concentrated in the transition from O0 to O1, and O3 -march=native did not alter the performance on GCC and was irregular on Clang, and neither vectorized the formation of* X^TX *or* X^Ty*. NE coefficient error increased with the condition number squared, limited to 3% before its Cholesky decomposition failed when the number reached around* $5 \cdot 10^7$*, while QR's error increased linearly with the number, limited to* $3 \cdot 10^{-9}$*. Residuals on all resolutions were practically identical.*

Keywords: *Linear Regression, compiler optimizations, OpenBLAS, Normal Equations, QR decomposition.*

REFERÊNCIAS

AMD. Zen 5 Delivers a New Level of Balanced Throughput. In: HOT CHIPS 2024. 2024. Disponível em: https://hc2024.hotchips.org/assets/program/conference/day2/24_HC2024.AMD.Cohen.Subramony.final.pdf . Acesso em: 2 nov. 2026.

BABOULIN, Marc; GIRAUD, Luc; GRATTON, Serge; LANGOU, Julien. Parallel tools for solving incremental dense least squares problems: application to space geodesy. Journal of Algorithms & Computational Technology, v. 3, n. 1, p. 117–140, 2009.

BLACKMAN, David; VIGNA, Sebastiano. Scrambled linear pseudorandom number generators. ACM Transactions on Mathematical Software, v. 47, n. 4, art. 36, p. 1–32, 2021.

CHANG, Xiao-Wen; KANG, Peng; TITLEY-PELOQUIN, David. Error bounds for computed least squares estimators. Linear Algebra and its Applications, v. 586, p. 28–42, 2020.

CHEIN, Flávia. Introdução aos modelos de regressão linear: um passo inicial para compreensão da econometria como uma ferramenta de avaliação de políticas públicas. Brasília, DF: Escola Nacional de Administração Pública (Enap), 2019. 76 p. ISBN 978-85-256-0115-5.

FENG, Jing Ge; HE, Ye Ping; TAO, Qiu Ming. Evaluation of Compilers' Capability of Automatic Vectorization Based on Source Code Analysis. Scientific Programming, v. 2021, art. 3264624, 2021.

GNU PROJECT. Optimize Options (Using the GNU Compiler Collection (GCC)). Disponível em: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html. Acesso em: 22 out. 2026.

GOLUB, Gene H.; VAN LOAN, Charles F. Matrix Computations. 4. ed. Johns Hopkins University Press, 2013.

HIGHAM, Nick. What is a Condition Number? Nick Higham, 19 mar. 2020a. Disponível em: https://nhigham.com/2020/03/19/what-is-a-condition-number/. Acesso em: 29 out. 2026.

HIGHAM, Nick. What is Numerical Stability? Nick Higham, 4 ago. 2020b. Disponível em: https://nhigham.com/2020/08/04/what-is-numerical-stability/. Acesso em: 29 out. 2026.

INSTITUTE OF ELECTRICAL AND ELECTRONICS ENGINEERS (IEEE). IEEE Standard for Floating-Point Arithmetic. New York: IEEE, 2019. IEEE Std 754-2019.

JETBRAINS. The State of C 2025. 2025. Disponível em: https://lp.jetbrains.com/the-state-of-c-2025/ . Acesso em: 31 out. 2026.

KERRISK, Michael. mallopt(3) — set memory allocation parameters. Linux manual page, 8 fev. 2026. Disponível em: https://man7.org/linux/man-pages/man3/mallopt.3.html . Acesso em: 8 nov. 2026.

LAPACK. DGELS: solves overdetermined or underdetermined systems for GE matrices. Netlib. Disponível em: https://netlib.org/lapack/lapack_routine/dgels.f . Acesso em: 31 out. 2026.

LARABEL, Michael. GCC 16 produces faster binaries than GCC 15, competitive race with LLVM Clang 22. Phoronix, 13 maio 2026. Disponível em: https://www.phoronix.com/review/gcc-16-vs-clang-22 . Acesso em: 2 nov. 2026.

LLVM PROJECT. Clang: the Clang C, C++, and Objective-C compiler. Disponível em: https://clang.llvm.org/docs/CommandGuide/clang.html . Acesso em: 1 nov. 2026.

OPENBLAS. Developer Manual. OpenMathLib. Disponível em: https://www.openmathlib.org/OpenBLAS/docs/developers/ . Acesso em: 1 nov. 2026.

R CORE TEAM. R: A language and environment for statistical computing. R Documentation: Fitting Linear Models. Disponível em: https://stat.ethz.ch/R-manual/R-devel/library/stats/html/lm.html . Acesso em: 31 out. 2026.

STEELE JR., Guy L.; LEA, Doug; FLOOD, Christine H. Fast splittable pseudorandom number generators. In: ACM SIGPLAN INTERNATIONAL CONFERENCE ON OBJECT-ORIENTED PROGRAMMING, SYSTEMS, LANGUAGES, AND APPLICATIONS, 2014, Portland. Proceedings [...]. New York: ACM, 2014. p. 453–472.

THOMPSON, Neil C.; GREENEWALD, Kristjan; LEE, Keeheon; MANSO, Gabriel F. The computational limits of deep learning. arXiv, 2020. Disponível em: https://arxiv.org/abs/2007.05558. Acesso em: 20 out. 2026.

AGRADECIMENTOS

Agradeço ao meu orientador, o professor Roger Rocha Ferreira, pela compreensão e atenção durante as reuniões, e aos meus pais e minha amada, pelo apoio ao longo do trabalho.